

A good practice is to periodically audit what you're collecting and ask, when did we last query this. If a metric has not been looked at in six months, it's probably safe to stop collecting. The same goes for logs and traces.

Common patterns that catch people out

There are a few mistakes I've watched teams make repeatedly when building out observability.

Alerting on causes instead of symptoms. CPU at 90 percent is a cause. Users seeing slow page loads is a symptom. The on-call engineer should be paged for the symptom. The cause is something they investigate after waking up. Pages on causes generate enormous noise because causes don't always lead to symptoms. A box can sit at 90 percent CPU all day with nobody noticing, because the service is still fast.

Building too many dashboards. Every team adds dashboards. Few teams remove them. Within a year, you have hundreds of dashboards, of which maybe ten are actually used. Be ruthless. Delete what isn't used. Mark canonical dashboards clearly so people don't get lost.

Treating observability as an afterthought. The teams that struggle most are the ones that try to add observability to a system that wasn't built with it in mind.

Confusing logging with observability. Just because you have a log aggregator doesn't mean you have observability. Logs are part of the picture. Metrics and traces complete it. A system with brilliant logging and no metrics is going to struggle to answer basic questions like "how busy is this service right now."

Where the field is moving

A few things worth keeping an eye on.

OpenTelemetry has won as the standard for emitting telemetry. If you're starting a new project today, instrument it with OpenTelemetry and you'll be able to swap backends without changing your

application code. That portability used to be hard. Now it's the default.

eBPF is changing what observability can see. Tools like Pixie, Parca, and Cilium Hubble can see inside running applications without modifying them at all. Network calls, function executions, system calls, all visible. It's still maturing, but the visibility is genuinely new.

AI-assisted incident response is starting to actually work. Tools that ingest your telemetry and either help you investigate, suggest likely root causes, or auto-summarise incidents are getting useful. Not magic, but useful.

Cost-aware observability is becoming a discipline of its own. As organisations realise how much they're spending on telemetry, tools that help you understand the cost-value of each metric, log, or trace are emerging. Expect this to become a more prominent topic in observability conversations.

What a sensible starting point looks like

If your organisation is just starting to build out observability seriously, here's a path I'd suggest.

Start with metrics. Install Prometheus. Get Node Exporter on every server. Get your applications to emit basic metrics about request volume, error rates, and latency. Build a dashboard that shows the four golden signals for each major service. This alone gives you 80 percent of the value with 20 percent of the effort.

Add structured logging next. Switch your applications to emit JSON logs. Ship them to a central log store, whether that's Loki, Elastic, or a managed service. Add request IDs to every log so you can correlate events later.

Then introduce tracing. Instrument your services with OpenTelemetry. Get

traces into Jaeger or Tempo. Train your team to read trace waterfalls. The day someone uses traces to find a latency problem that would have taken hours with logs alone is the day everyone becomes a believer.

Finally, formalise your SLOs. Pick one or two for your most critical user journeys. Track them. Use them to drive prioritisation conversations. This shifts how the team thinks about reliability.

You don't have to do all of this at once. You don't have to do it perfectly. The teams I admire most are the ones who iterated on their observability for years and ended up with something genuinely powerful, not the ones who tried to design it all up front and got stuck in analysis paralysis.

The quiet work that keeps things running

The truth nobody puts on a billboard is that a modern data centre is held together by an enormous amount of careful, unglamorous, mostly invisible work. The dashboards nobody screenshots. The alerts that fire and resolve themselves before anyone notices. The runbooks updated after an incident. The weird metric somebody added two years ago that turned out to save a Friday evening. None of it makes the keynote. All of it matters. If you ever find yourself wondering why your favourite streaming service worked perfectly last night despite a regional outage, or why your bank's app didn't blink during a record traffic spike, the answer is almost always the same. Somewhere, a team that you'll never meet built the observability that caught the problem early and acted before it got loud. Take a moment to appreciate that. Then go build a little more of it yourself. **END** 🐧

 By: Dhaval Gajjar

The author is the CTO of Textdrip and the CEO of Pranshtech. He spends his days building and scaling production systems and writes occasionally about the messy, unglamorous side of keeping services alive..